Mitigation Methods for Class Imbalance in Histopathology Patch Classification

Yannik Queisler

October 11, 2026

Abstract

Class imbalance lets a patch classifier reach high aggregate accuracy while failing on rare diagnoses. This report defines six mitigation methods for histopathology patch classification and places them in the data-level, algorithm-level, and hybrid families of Krawczyk (2016): balanced sampling and a curriculum of data augmentation (CUDA) change the training data, logit adjustment and DisAlign change the objective or decision rule, and classifier retraining (cRT) and Gaussian clouded logit (GCL) adjustment combine both. All methods share one setting: a Virchow2 foundation-model encoder that is either frozen or adapted with low-rank updates, and a linear classification head. Each method is stated in its source form, with the control parameter that sets its strength and the adaptations required by this setting.

Contents

1 Introduction **2**

2 Taxonomy **2**

3 Methods **3**

3.1 Cross-Entropy Reference 3

3.2 Data-Level Methods 3

3.3 Algorithm-Level Methods 4

3.4 Hybrid Methods 6

4 Summary **7**

1 Introduction

Diagnostic datasets in pathology are rarely balanced. Common tumour types and benign tissue dominate the training material, while the classes a diagnostic system is often meant to catch appear only a few times. A classifier trained on such data can reach high overall accuracy while its recall on rare classes stays low, because prevalence-weighted metrics reward the classes that already dominate. Long-tailed learning and medical-image evaluation literature therefore treat classwise recall, balanced accuracy, and calibration-aware probability metrics as primary outcomes under imbalance (Saini and Susan, 2023; Zhang et al., 2023; Mosquera et al., 2024). Mitigation methods try to raise tail-class performance without giving up the head classes, and they differ in where they intervene. This report fixes the definitions of six such methods, so that later experiments can compare them under one protocol.

Setting. All methods are defined for patch classification. Each labelled patch x is embedded by the Virchow2 encoder f_θ, a ViT-H/14 foundation model pretrained on pathology images (Zimmermann et al., 2024), and a linear head maps the embedding to class logits $\mathbf{z} = W f_\theta(x) + \mathbf{b}$. The broader pipeline around this setting, from whole-slide imaging to patch extraction, is described in the companion datasets report (Queisler, 2026). Methods whose source form learns the representation may adapt the encoder, but only through lightweight updates. Low-rank adaptation (LoRA) keeps each pretrained weight matrix W_0 frozen and learns an additive update $\Delta W = BA$ of small rank r, applied to the attention projections (Hu et al., 2022). Only A, B, and the head are trained, and the rest of the encoder remains fixed. This follows the evidence on adapting large pretrained models. Full fine-tuning can distort pretrained features that already generalize (Kumar et al., 2022). Under long-tailed data, heavy fine-tuning of a foundation model lowers tail-class accuracy, while lightweight fine-tuning keeps the class-conditional features consistent and performs better (Shi et al., 2024). The labelled patch sets are also small relative to the roughly 632 million encoder parameters, and small trainable adapters on frozen pathology encoders are an established way to limit overfitting in this regime (Hasan et al., 2025). Three methods train in two stages: cRT, DisAlign, and GCL. For these, stage one learns the encoder adaptation and the head, and stage two freezes the encoder together with its LoRA update and trains only the classifier or calibration parameters.

Notation Let C be the number of classes and n_c the number of training patches of class c, so that $N = \sum_{j=1}^{C} n_j$ is the training-set size and $\pi_c = n_c/N$ is the empirical prevalence of class c. Let $n_{\max} = \max_c n_c$. For a patch with label $y \in \{1, \dots, C\}$, the model outputs logits $\mathbf{z} \in \mathbb{R}^C$ and class probabilities

$$p_c = \frac{\exp(z_c)}{\sum_{j=1}^{C} \exp(z_j)}. \tag{1}$$

Unless a method changes the sampler, the objective, or the decision rule, training draws patches from the empirical class distribution.

2 Taxonomy

Krawczyk (2016) sorts imbalance methods into three families by what they modify. *Data-level* methods change the training set or the way it is sampled, so that a standard learner sees a more balanced or more varied distribution. *Algorithm-level* methods leave the data as observed and change the learner itself, through its loss, its output layer, or its decision rule, so that the learner is no longer biased toward majority classes. *Hybrid* methods combine a data-level and an algorithm-level change, so that neither component alone explains their effect. Cross-entropy (CE) on the empirical distribution is the reference outside all three families: it changes neither data nor learner.

Figure 1 places the six methods in these families. Two placements need a short justification. cRT and GCL are hybrid because both decouple representation learning from classifier learning, which is an algorithm-level change, and both rebalance the sampler in the classifier stage, which is a data-level change. DisAlign also trains in two stages, but its second stage keeps the sampler unchanged and rebalances only through a reweighted objective and a learned logit calibration, so it is algorithm-level.

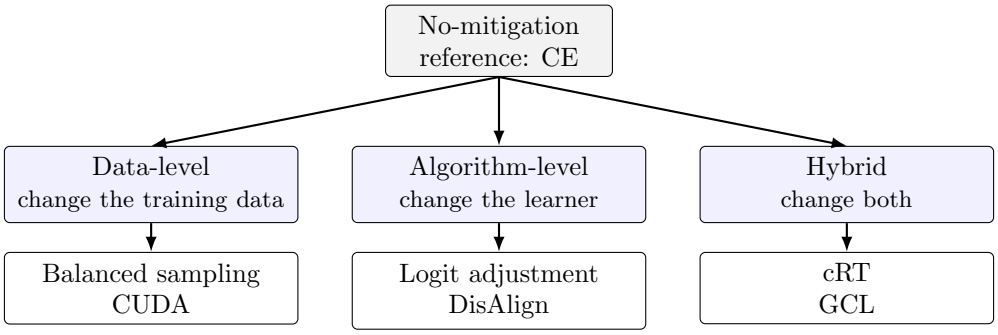


Figure 1: The six methods in the three families of Krawczyk (2016). CE on the empirical distribution is the no-mitigation reference.

3 Methods

This section defines the methods in the order of Figure 1. Each definition gives the source form first and then, where needed, the thesis variant: how the method is instantiated on the Virchow2 encoder with a linear head.

3.1 Cross-Entropy Reference

Unweighted CE is the standard supervised objective without an imbalance correction. For a patch with label y, the loss is

$$\mathcal{L}_{\mathrm{CE}}(\mathbf{z}, y) = -\log p_y. \tag{2}$$

Minibatches are drawn from the empirical distribution, so head classes contribute more loss terms per epoch than tail classes. CE is trained in the same setting as the other methods, with the same LoRA adaptation, so that each mitigation is compared with a reference that differs only in the mitigation itself.

3.2 Data-Level Methods

Data-level methods change what the optimizer sees and keep the CE objective. They are the simplest interventions to state, and they leave the model and its output semantics unchanged.

3.2.1 Balanced Sampling

Class-aware sampling fills each minibatch by first drawing a class uniformly and then drawing a patch from that class, so that every class appears at the same expected rate (Shen et al., 2016). Kang et al. (2020) write this family as a power of the class count. With strength $s \in [0, 1]$, patch i with label y_i is sampled with probability

$$q_i(s) = \frac{n_{y_i}^{-s}}{\sum_{k=1}^{N} n_{y_k}^{-s}}, \tag{3}$$

so that class c receives an expected share proportional to n_c^{1-s}. Setting $s = 0$ recovers empirical sampling and CE; $s = 1$ gives every class the same expected exposure; intermediate values rebalance partly. The loss stays Equation (2). Balanced sampling repeats tail patches rather than adding new ones, so at high s it can overfit the few tail patches, while the head classes are seen less often per epoch.

3.2.2 Curriculum of Data Augmentation (CUDA)

Curriculum of data augmentation (CUDA) assigns each class its own augmentation strength and adapts it during training (Ahn et al., 2023). It rests on an observation of the source: strong augmentation of one group of classes tends to raise the accuracy of the other classes, while the augmented classes themselves gain little or even lose accuracy. The appropriate strength therefore differs between classes, and CUDA sets it from how well the model has already learned each class rather than from the class counts.

The augmentation is built from K predefined image operations, such as rotation, shifts, or brightness changes. Each operation O_k has a magnitude function $m_k(s)$ of an integer strength $s \in \{0, \dots, S\}$. At strength s, the augmentation applies s operations drawn uniformly at random, each at magnitude $m_k(s)$:

$$O(x; s) = O_{k_s}^{m_{k_s}(s)} \circ \cdots \circ O_{k_1}^{m_{k_1}(s)}(x), \qquad k_i \sim \mathcal{U}\{1, \dots, K\}. \tag{4}$$

Strength $s = 0$ leaves the patch unchanged, and a larger s gives both more and stronger operations.

Each class c carries a *level of learning* L_c, which is the strength currently applied to its patches. All levels start at zero and are updated at the start of every epoch. For each candidate strength $l \in \{0, \dots, L_c\}$, a subset D'_c of $T(l+1)$ patches of class c is drawn with replacement, augmented at strength l, and classified by the current model. The level rises only if the model classifies enough of them correctly at every strength:

$$L_c \leftarrow \begin{cases} L_c + 1 & \text{if } \sum_{x \in D'_c} \mathbf{1}\{\hat{y}(O(x; l)) = c\} \geq \gamma\, T(l+1) \text{ for all } l \leq L_c, \\ L_c - 1 & \text{otherwise,} \end{cases} \tag{5}$$

where $\gamma \in [0, 1]$ is the accuracy threshold and T sets how many patches are checked; the check grows with l because the number of possible operation sequences grows with the strength. During the epoch, each training patch x of class y is replaced by $O(x; L_y)$ with probability p_{aug} and kept unchanged otherwise, and the loss stays Equation (2). The curriculum thus presents easy versions of a class first and harder ones once the class is learned.

CUDA does not read class counts, and the sampler stays empirical. Its effect on imbalance comes through the levels: head classes are learned sooner, reach higher levels, and receive stronger augmentation, which by the source's observation benefits the other classes. The augmentation probability $p_{\text{aug}} \in [0, 1]$ is the control: $p_{\text{aug}} = 0$ recovers CE, and larger values apply the curriculum to more patches. The source designs CUDA as an add-on to other long-tailed methods; here it is paired with CE, so that its effect is not mixed with a second rebalancing mechanism.

Thesis variant. The operations act on the image patch before the Virchow2 encoder, so CUDA acts in stage one together with the LoRA update. Training is counted in optimizer steps rather than epochs, so the levels are updated at a fixed step interval. The threshold γ, the check size T, the maximum strength S, and the operation set are fixed before training; p_{aug} is the tuned control.

3.3 Algorithm-Level Methods

Algorithm-level methods keep the empirical data stream and change how the learner turns features into decisions. Both methods below act on the logits: logit adjustment with a fixed shift derived from the class prior, DisAlign with a learned, input-dependent calibration.

3.3.1 Logit Adjustment

Logit adjustment corrects for the training prior π_c (Menon et al., 2021). A classifier trained with CE estimates the training posterior, which favours head classes in proportion to their prior. Removing that prior gives decisions that suit a class-balanced target. The strength $\tau \geq 0$ scales the correction, and $\tau = 0$ recovers CE. In the *train-time* form, the prior enters the loss:

$$\mathcal{L}_{\mathrm{LA}}(\mathbf{z}, y) = -\log \frac{\exp(z_y + \tau \log \pi_y)}{\sum_{c=1}^{C} \exp(z_c + \tau \log \pi_c)}. \tag{6}$$

Head classes receive a larger additive offset during training, so the model must learn a larger margin for tail classes to fit them; at inference the unadjusted logits z_c are used. In the *post-hoc* form, the model is trained with CE and the prior is removed only at inference:

$$\hat{y} = \arg\max_c \big(z_c - \tau \log \pi_c\big). \tag{7}$$

The sign differs because the train-time form learns against prior-shifted logits, while the post-hoc form subtracts the prior from ordinary CE logits. At $\tau = 1$ both target the balanced error. The post-hoc form needs no retraining and can therefore be applied to the CE model of every other method.

Either form changes the implied class prior, so the resulting probabilities are not calibrated for the natural test prevalence. Discrimination and calibration are therefore reported separately, and temperature scaling is kept distinct from prior correction.

3.3.2 DisAlign

Distribution alignment (DisAlign) keeps a CE-trained classifier and learns a light correction of its scores (Zhang et al., 2021). Stage one trains the model with CE. Stage two freezes the representation and the original head, whose logits are z_j^o, and learns for each class a scale α_j and a bias β_j. An input-dependent confidence $\sigma(x) = \mathrm{sigmoid}(\mathbf{v}^\top f_\theta(x))$ decides how much of the correction is applied:

$$\hat{z}_j = \sigma(x)\big(\alpha_j z_j^o + \beta_j\big) + \big(1 - \sigma(x)\big) z_j^o = \big(1 + \sigma(x)\alpha_j\big) z_j^o + \sigma(x)\beta_j. \tag{8}$$

Only α_j, β_j, and $\mathbf{v}$ are trained, which is far fewer parameters than a new classifier. Post-hoc logit adjustment is the special case with unit scale and a fixed bias $-\tau \log \pi_j$, so DisAlign can be read as a learned, input-dependent generalization of it.

The calibration is fitted by aligning the model prediction with a reference distribution that favours balanced prediction. The reference puts weight w_c on the true class of each training patch, with the generalized reweighting

$$w_c(\rho) = \frac{\pi_c^{-\rho}}{\sum_{k=1}^{C} \pi_k^{-\rho}}, \qquad \mathcal{L}_{\mathrm{DisAlign}} = -\frac{1}{N} \sum_{i=1}^{N} w_{y_i}(\rho) \log \hat{p}_{y_i}(x_i), \tag{9}$$

where $\hat{p}$ is the softmax of the calibrated logits $\hat{\mathbf{z}}$. The exponent $\rho \geq 0$ is the control: $\rho = 0$ weights every patch equally, $\rho = 1$ gives inverse-prevalence class weights, and larger values emphasize the tail further. Stage two keeps the empirical sampler, so the rebalancing enters only through w_c.

Thesis variant. Stage one is the CE reference with LoRA adaptation. Stage two freezes the encoder, its LoRA update, and the linear head, and trains only the calibration parameters of Equation (8). The source also reports a cosine-classifier variant; the tested variant keeps the linear head of the reference.

3.4 Hybrid Methods

Both hybrid methods follow the decoupling view of long-tailed learning: the representation is learned in a first stage, and a class-rebalanced sampler then corrects the classifier in a second stage in which the representation is frozen. They differ in how the first stage is trained and in how strongly the second stage rebalances.

3.4.1 Classifier Re-Training (cRT)

Classifier re-training separates representation learning from classifier fitting (Kang et al., 2020). Stage one trains the model with CE on the empirical distribution. Stage two freezes the learned representation $h(x)$, discards and reinitializes the linear classifier, and fits it with class-balanced sampling:

$$(\widehat{W}, \widehat{\mathbf{b}}) = \arg\min_{W,\mathbf{b}} \; \mathbb{E}_{(x,y)\sim q_{\text{bal}}}\Big[-\log \operatorname{softmax}\big(Wh(x)+\mathbf{b}\big)_y\Big], \tag{10}$$

where q_{bal} is Equation (3) at $s = 1$. The premise is that instance-balanced training learns the best representation, and that imbalance mainly biases the classifier. cRT has no strength parameter; its rebalancing is complete by construction.

Thesis variant. The representation is $h(x) = f_\theta(x)$ with the LoRA update learned in stage one. The source also studies classifier normalization variants; the tested variant uses only the reinitialized linear classifier.

3.4.2 Gaussian Clouded Logit (GCL) Adjustment

GCL adjustment addresses two effects of long-tailed training in separate stages (Li et al., 2022). In the first stage, the embedding space is distorted: tail classes occupy a small region, and softmax saturation stops their gradients early, once the target logit is well ahead. GCL counters this by perturbing the logits with noise whose amplitude grows toward the tail. With a cosine classifier, logits are $s\cos\theta_j$, where θ_j is the angle between the feature and the class weight $\mathbf{w}_j$, and s is a fixed scale. The clouded logit of class j is

$$\tilde{z}_j = s\big(\cos\theta_j - \delta_j\,|\varepsilon|\big), \qquad \delta_j = \log n_{\max} - \log n_j, \qquad \varepsilon \sim \mathcal{N}(0,\sigma^2), \tag{11}$$

and the model is trained with CE on $\tilde{\mathbf{z}}$. The most frequent class receives no noise, and tail classes receive the largest cloud size δ_j. The noise always lowers the logit, so a tail patch must lie further from the class boundary before its gradient saturates; this keeps tail patches active in training and enlarges their margin. The noise scale σ is the control: $\sigma = 0$ recovers CE with a cosine classifier. At inference no noise is added.

In the second stage, the representation is frozen and the classifier is re-trained with a class-rebalanced sampler. Because GCL already lets tail patches contribute more in stage one, fully balanced sampling would over-train them. GCL instead samples class j with the class-based effective number of Cui et al. (2019),

$$r_j = \frac{1-\beta_j}{1-\beta_j^{n_j}}, \qquad \beta_j = b\,\frac{\delta_j - \delta_{\min}}{\delta_{\max} - \delta_{\min}} + a, \tag{12}$$

normalized over classes, where a and b set the range $[a, a+b]$ of β_j. A larger cloud size gives a larger β_j and hence a larger effective-number correction, so tail classes are sampled more often.

The reference implementation sets $a = 0.9999$ and $b = 10^{-5}$, so $1/(1-\beta_j) \approx 10^4$ exceeds every class count, $r_j \approx 1/n_j$, and the sampler is close to class-balanced; the head's effective number saturates first, which leaves it slightly over-weighted relative to the tail. The stage-two loss is again the clouded loss of Equation (11).

Thesis variant. Stage one trains the LoRA update and a cosine classifier head with the clouded loss. Stage two freezes the encoder and its LoRA update and re-trains only the cosine head with the sampler of Equation (12). The scale $s = 30$ and the range $a = 0.9999$, $b = 10^{-5}$ (reference implementation) are fixed before training; σ is the tuned control. GCL replaces the linear head of the reference with a cosine head, so part of its effect may come from the head geometry rather than from the logit noise.

4 Summary

This report defines six class-imbalance mitigation methods for patch classification and places them in the three families of Krawczyk (2016). Balanced sampling and CUDA are data-level methods, logit adjustment and DisAlign are algorithm-level methods, and cRT and GCL are hybrid methods that pair decoupled classifier learning with a rebalanced sampler. All methods except CUDA read the imbalance from the class counts n_c or the prevalences π_c; CUDA reads it from the per-class accuracy of the current model, and each method except cRT has one parameter that sets its strength. At zero strength, balanced sampling, CUDA, and logit adjustment reduce to CE, and GCL reduces to CE with a cosine head. The methods share one setting: a Virchow2 encoder adapted only through LoRA, a linear or cosine head, and, for the three two-stage methods, a frozen encoder in stage two. Later experiments compare them against CE with balanced accuracy, classwise recall, and probability quality, with raw and temperature-scaled outputs reported separately.

References

Sumyeong Ahn, Jongwoo Ko, and Se-Young Yun. CUDA: Curriculum of data augmentation for long-tailed recognition. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2302.05499.

Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. Class-balanced loss based on effective number of samples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9260–9269, 2019. doi: 10.1109/CVPR.2019.00949.

Kazi Rakib Hasan, Sijin Kim, Junghwan Cho, and Hyung Soo Han. Prototypical few-shot learning for histopathology classification: Leveraging foundation models with adapter architectures. *IEEE Access*, 13:86356–86379, 2025. doi: 10.1109/ACCESS.2025.3570673.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

Bingyi Kang, Saining Xie, Marcus Rohrbach, Zhicheng Yan, Albert Gordo, Jiashi Feng, and Yannis Kalantidis. Decoupling representation and classifier for long-tailed recognition. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=r1gRTCVFvB.

Bartosz Krawczyk. Learning from imbalanced data: open challenges and future directions. *Progress in Artificial Intelligence*, 5(4):221–232, 2016. doi: 10.1007/s13748-016-0094-0.

Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=UYneFzXSJWh.

Mengke Li, Yiu ming Cheung, and Yang Lu. Long-tailed visual recognition via Gaussian clouded logit adjustment. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6919–6928, 2022. doi: 10.1109/CVPR52688.2022.00680.

Aditya Krishna Menon, Sadeep Jayasumana, Ankit Singh Rawat, Himanshu Jain, Andreas Veit, and Sanjiv Kumar. Long-tail learning via logit adjustment. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=37nvvqkCo5.

Candelaria Mosquera, Luciana Ferrer, Diego H. Milone, Daniel Luna, and Enzo Ferrante. Class imbalance on medical image classification: towards better evaluation practices for discrimination and calibration performance. *European Radiology*, 34(12):7895–7903, 2024. doi: 10.1007/s00330-024-10834-0.

Yannik Queisler. Computational pathology and benchmark datasets: An introduction to the domain and data. Technical report, TU Berlin, 2026.

Manisha Saini and Seba Susan. Tackling class imbalance in computer vision: a contemporary review. *Artificial Intelligence Review*, 56(S1):1279–1335, 2023. doi: 10.1007/s10462-023-10557-6.

Li Shen, Zhouchen Lin, and Qingming Huang. Relay backpropagation for effective learning of deep convolutional neural networks. In *European Conference on Computer Vision*, pages 467–482, 2016. doi: 10.1007/978-3-319-46478-7_29.

Jiang-Xin Shi, Tong Wei, Zhi Zhou, Jie-Jing Shao, Xin-Yan Han, and Yu-Feng Li. Long-tail learning with foundation model: Heavy fine-tuning hurts. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 45014–45039, 2024. URL `https://proceedings.mlr.press/v235/shi24g.html`.

Songyang Zhang, Zeming Li, Shipeng Yan, Xuming He, and Jian Sun. Distribution alignment: A unified framework for long-tail visual recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2361–2370, 2021. doi: 10.1109/CVPR46437.2021.00239.

Yifan Zhang, Bingyi Kang, Bryan Hooi, and Shuicheng Yan. Deep long-tailed learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(9):10795–10816, 2023. doi: 10.1109/TPAMI.2023.3268118.

Eric Zimmermann, Eugene Vorontsov, Julian Viret, Adam Casson, Michal Zelechowski, George Shaikovski, Neil Tenenholtz, James Hall, Thomas Fuchs, Nicolo Fusi, Siqi Liu, and Kristen Severson. Virchow2: Scaling self-supervised mixed magnification models in pathology. *arXiv preprint arXiv:2408.00738*, 2024. doi: 10.48550/arXiv.2408.00738.